

Group Topic: **Big Data Analysis for
Rainfall-Based Flood Risk Prediction in
Albay using SYNOP Data**

Week 4

Git Repo/ Colab Link:
<https://github.com/PotatoKevz/Big-Data-Analysis-Repository>

Date: May 15, 2026

Objectives :

The main goal of this phase is to improve the flood risk prediction system by:

- Developing a modular machine learning pipeline integrated with external tools for better functionality and usability.
- Creating professional and interactive visualizations for processed data and model outputs.
- Presenting clear algorithm performance metrics and visual outputs to show the progress of the project.

Introduction :

This laboratory activity aims to further improve the students' practical skills in handling big data by developing a modular machine learning pipeline integrated with external tools. It also builds upon the knowledge and skills gained from the previous laboratory activities related to big data processing and analysis.

The activity focuses on implementing a Flood Risk Prediction System that analyzes SYNOP weather station data collected from multiple stations surrounding Albay. The system uses machine learning models to study rainfall patterns and predict possible flood risk events. Several technologies were integrated into the system, including Python-based data processing libraries, machine learning algorithms, and an interactive visualization dashboard.

As a result, the project provides a complete platform that transforms raw weather data into meaningful insights that may support disaster preparedness and decision-making processes.

Methodology :

The activity catered to a data-driven classification approach to process data using historical SYNOP meteorological data, a basis for the project objective, which is to predict rainfall-based flood risk. The methodology was executed through a structured, end-to-end pipeline, and has also implemented changes from the previous activity's methods. Due to these, some approaches from this activity were slightly modified from the earlier task documentations. This week's activity involved primary steps that were done to accomplish; developing a pipeline that utilizes external tools, and visualizing processed data in a professional format.

1. Pipeline Development with External Tool Integration
 - A core backend pipeline (`flood_risk_pipeline.py`) was developed to handle data loading, preprocessing, feature engineering, model training, and prediction generation.
 - The pipeline outputs (predictions, metrics, and model artifacts) were designed to be easily consumed by external applications.
 - Streamlit was integrated as the primary external tool to build an interactive web-based dashboard (`dashboard.py`).
 - Additional external libraries integrated include:
 - **Plotly** – for creating interactive and dynamic visualizations.
 - **Joblib** – for saving and loading the trained machine learning model (`rf_flood_model.pkl`).
 - **Pandas** – for efficient data loading and manipulation between pipeline and dashboard.
 - This integration established a complete workflow:

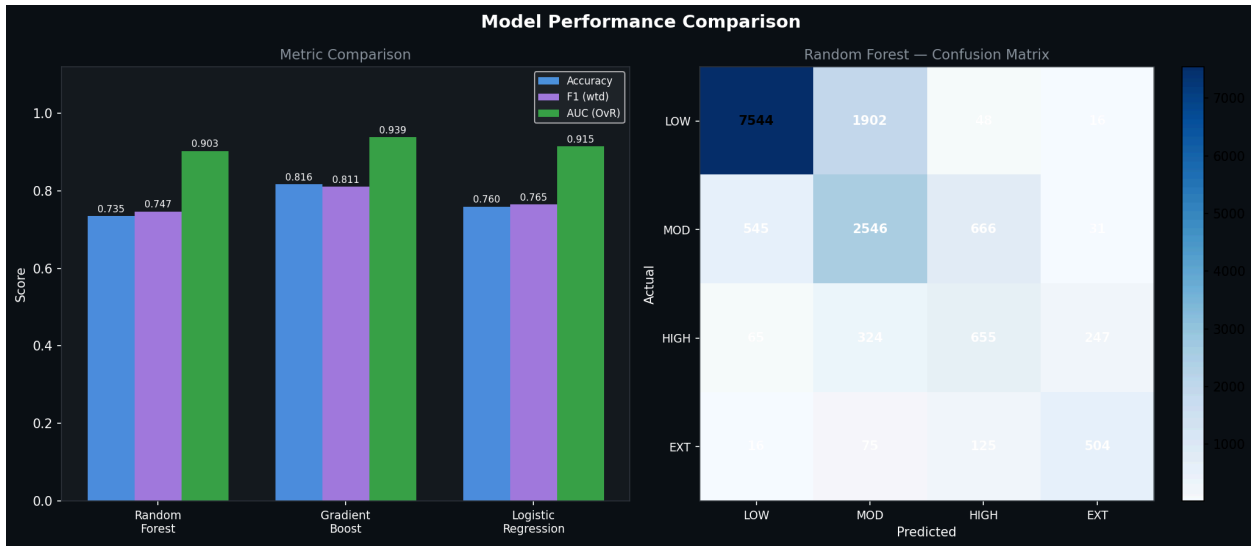
SYNOP Data → Pipeline Processing → Model Prediction → Streamlit Dashboard.

2. Professional Data Visualization
 - All visualizations were generated using **Plotly** within the Streamlit dashboard to ensure high interactivity and professional quality.
 - A clean, multi-tab dashboard layout was implemented with three main sections:
 - **Overview Tab**: Time series plots, risk class pie charts, rainfall histograms, and KPI cards.
 - **Model Performance Tab**: Comparative bar charts for algorithm metrics and feature importance visualizations.
 - **Extreme Events Tab**: Scatter plots highlighting high-risk and extreme rainfall occurrences.
 - Interactive features such as date range filters, risk class selectors, hover tooltips, zoom/pan capabilities, and conditional formatting were added to enhance usability.
 - A professional dark theme with consistent, accessible colors was applied throughout the dashboard.

Results and Analysis :

The system evaluated three machine learning models for flood and risk prediction. The results showed that the Gradient Boosting model achieved the best overall performance across multiple evaluation metrics.

Model	Accuracy	Weighted F1	AUC
Random Forest	73.48%	0.7469	0.903
Gradient Boost	81.64%	0.8107	0.939
Logistic Regression	75.96%	0.7654	0.915



The Gradient Boosting model achieved the highest accuracy of 81.64% and the highest AUC score of 0.939, indicating strong predictive capability and a good balance between precision and recall across flood risk categories.

This model was therefore selected as the best-performing algorithm from the flood prediction system.

Feature Importance Analysis

It revealed that rainfall-related features played the most significant role in predicting flood risk. The most influential predictors included:

- Rainfall lag features from Albay station
- 24-hour rainfall accumulation
- Rainfall data from nearby stations such as Catanduanes

These results indicate that recent rainfall patterns and upstream precipitation are key indicators of flood risk.

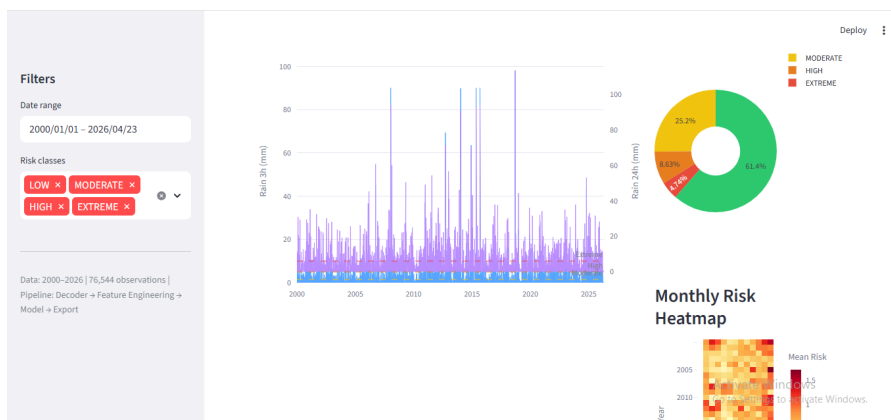
VISUALIZATION RESULTS

The system produced both interactive visualizations and static charts for analysis and reporting.

Interactive Dashboard Visualizations

Overview Tab

- Rainfall timeline showing precipitation levels over time
- Flood risk distribution pie chart

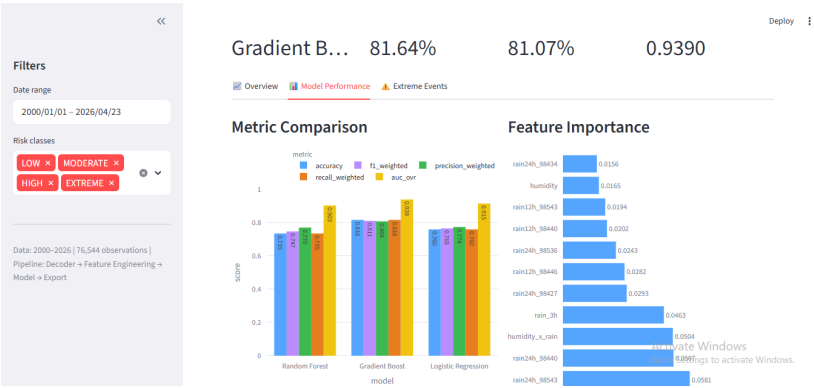


- Monthly risk heatmap
- Key performance indicator (KPI) cards showing high and extreme risk events

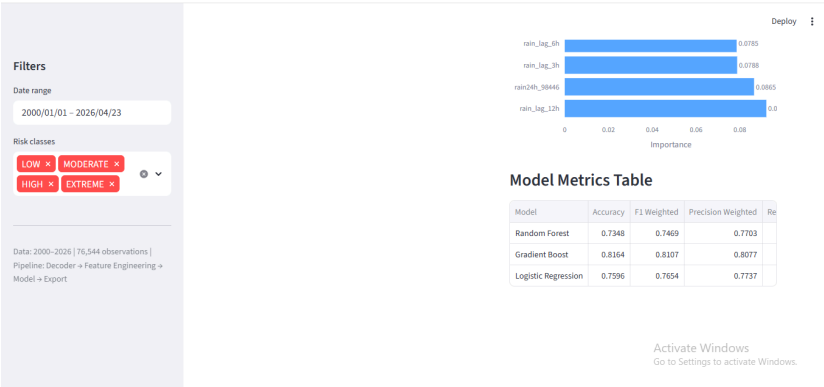


Model Performance Tab

- Grouped bar chart comparing performance of all models
- Feature importance charts displaying top predictive variables

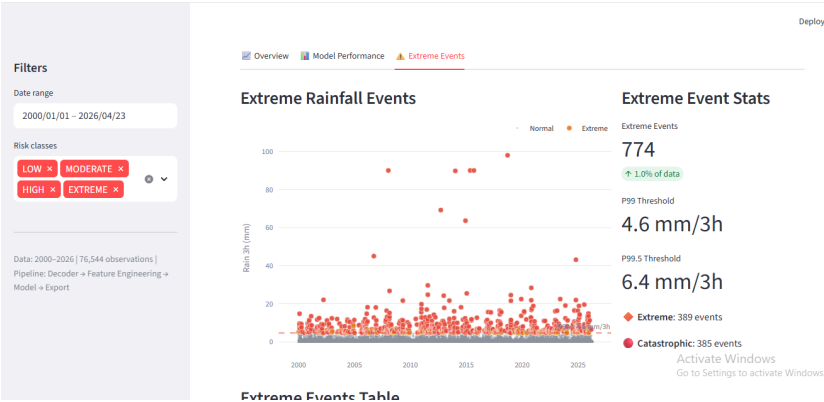


- Sortable metrics table for deeper analysis

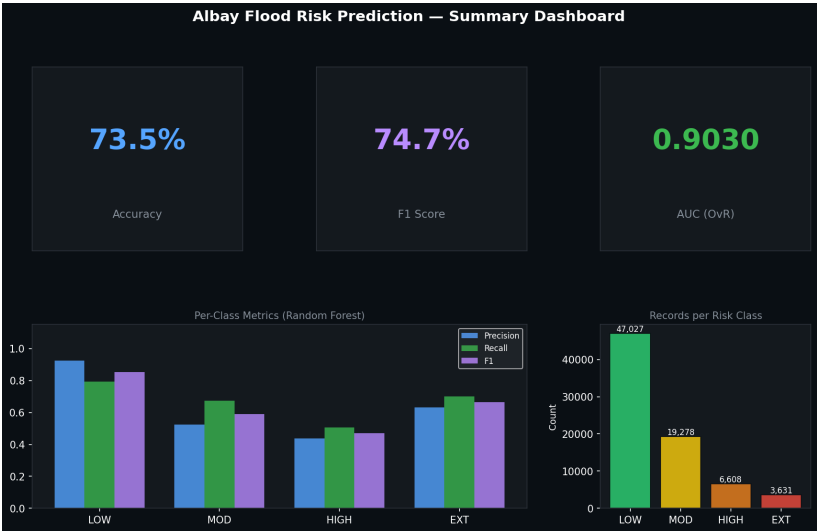


Extreme Events Tab

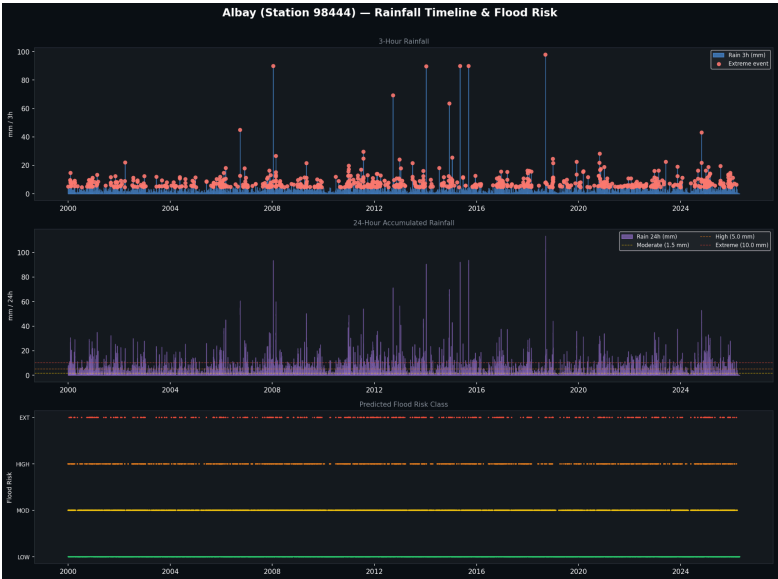
- Scatter plot highlighting extreme rainfall events
- Statistical summaries of high-risk occurrences



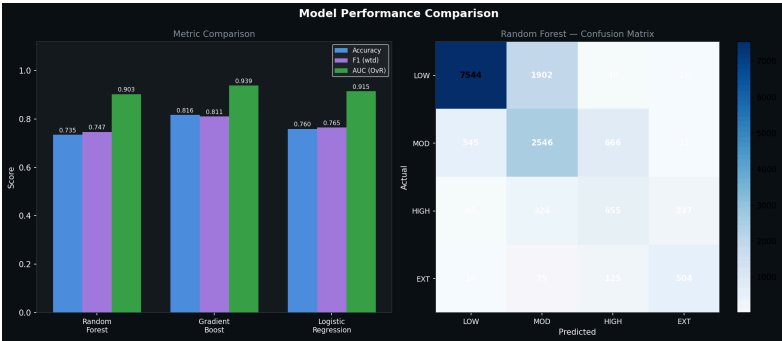
- Summary dashboard visualization



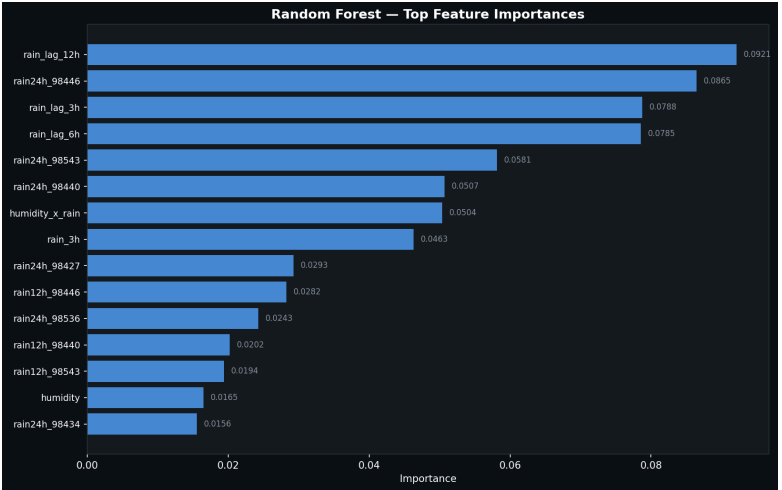
- Rainfall timeline chart



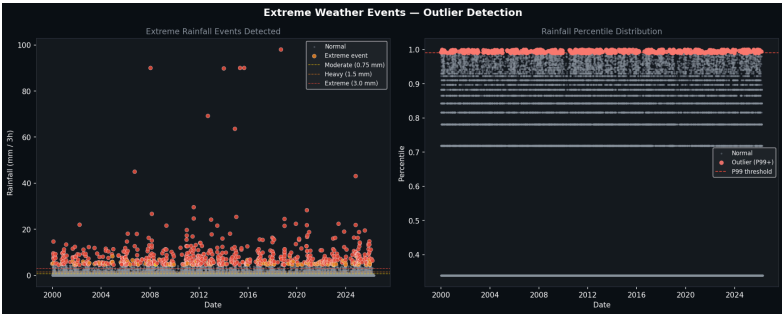
- Model comparison chart



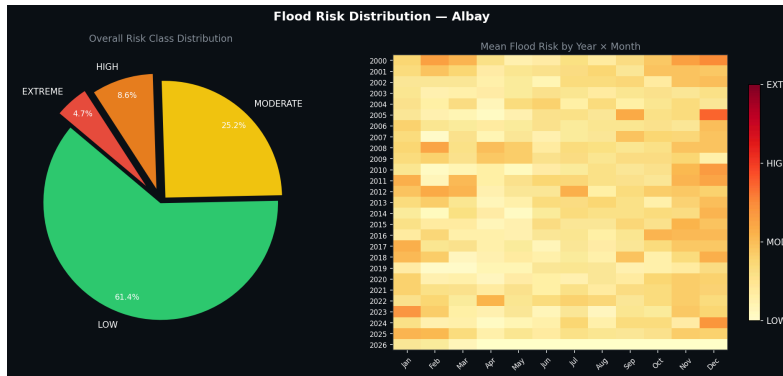
- Feature importance chart



- Extreme events scatter plot



- Risk distribution chart



These static charts provide a permanent record of the analysis and can be used in reports or presentations.

Challenges and Solutions :

During the implementation of the pipeline development and professional visualization tasks, the group encountered several technical and practical challenges. These challenges arose from the earlier methods and steps taken when accomplishing previous tasks. Below are the major ones identified along with the solutions applied:

Challenge 1: Target Leakage in Model Training

- Issue: Future rainfall features such as `rain_24h` and `rain_48h` were mistakenly included in the feature set, resulting in unrealistically high model performance.
- Solution: All future-dependent features were reviewed and removed from the training dataset. Only past and current observations were retained for prediction. Risk labels were generated strictly from the target 24-hour rainfall after feature engineering.

Challenge 2: Incorrect Rainfall Encoding (`rrr` field)

- Issue: Rainfall values in the SYNOP decoder were scaled 10 times higher than the actual values, leading to incorrect risk classifications and misleading visualizations.
- Solution: The rainfall decoding logic in `synop_decoder.py` was corrected and the entire dataset was reprocessed. Risk thresholds were also adjusted to align with DOST-PAGASA standards.

Challenge 3: Multi-Station Data Alignment

- Issue: Supporting stations contained inconsistent timestamps and missing readings, making it difficult to properly merge the data with the main Legazpi station (98444).

- Solution: A 3-hour resampling method (`.resample('3h').mean()`) was implemented together with forward and backward filling, including proper missing-data flags for supporting stations.

Challenge 4: Class Imbalance and Dashboard Bias

- Issue: The dataset was heavily dominated by LOW risk cases (approximately 67%), causing model bias toward the majority class and affecting visualization interpretation.
- Solution: The pipeline applied `class_weight='balanced'` and stratified sampling techniques. Additional KPI cards and percentage breakdowns were also added to the dashboard to clearly show the imbalance.

Challenge 5: Dashboard Integration and Crashes

- Issue: The Streamlit dashboard crashed while loading outputs because of mismatched feature names between the pipeline and saved CSV files.
- Solution: Output file formats were standardized, while additional error handling and data validation were implemented in `dashboard.py`. Consistent column naming was also enforced between the pipeline and dashboard.

Challenge 6: Creating Professional and Interactive Visualizations

- Issue: Basic Matplotlib charts lacked interactivity and were not visually polished enough for presentations.
- Solution: Plotly was integrated within Streamlit to create interactive charts with zoom and hover functionalities. A clean three-tab dashboard layout with filters, dark theme, and conditional formatting was also implemented to improve presentation quality.

Conclusion :

The laboratory activity successfully demonstrated the use of machine learning and data visualization tools in analyzing rainfall data and predicting possible flood risks. Through the integration of data processing, predictive modeling, and an interactive dashboard, the system was able to provide a complete solution for analyzing flood risk patterns in Albay.

Among the evaluated models, the Gradient Boosting algorithm achieved the best overall performance, making it the most reliable predictor within the system. The visualization tools also improved the usability of the project by presenting complex analytical outputs in a more understandable and accessible format.

Overall, the project highlights the potential of data-driven approaches in supporting disaster preparedness and environmental monitoring. Future improvements may include adding more environmental variables, expanding the dataset to include additional weather stations, and deploying the system for real-time flood monitoring.